Sésame

Let Claude book with any merchant, straight from their website.

BaselHack 2026 · restaurants first

THE PROBLEM

Claude can find a restaurant. It can't book a table there.

1

A website and a phone

Most small restaurants take bookings by phone, by email or through a form on their own site.

2

Nothing an assistant can call

No API and no tools. An assistant can only send the guest to the site and stop there.

3

No time to integrate

A restaurant won't build and run a server so that assistants can book with it.

THE IDEA

A website and two sentences in. A booking agent out.

In

The merchant chats

The website's address and how the room works: `44 seats on 12 tables, groups up to 6`.

Built

Sésame reads and builds

It reads the site and fills a validated profile with the hours, tables and rules. It then sets up the restaurant's own data and its booking tools.

Out

Claude books, the merchant sees it

Guests book from Claude through one connector. Each booking appears live on the merchant's management site.

LIVE DEMO · 5 MINUTES

Le Petit Zinc, a fictional bistro in Mulhouse

0:00 · Merchant

Its own website: menu, hours, a booking form

0:30 · Merchant

Studio reads it and builds the agent, live. One correction, then Publish

2:00 · Guest

“Table for 4 tonight at 8pm in Mulhouse”. Preferences filter, Claude asks, then books

3:00 · Merchant

The booking arrives live on the management site

3:30 · Merchant

Block 21:00. Claude stops offering it

4:00 · Guest

A table for 8: Claude hands over the restaurant's booking page

4:20 · Merchant

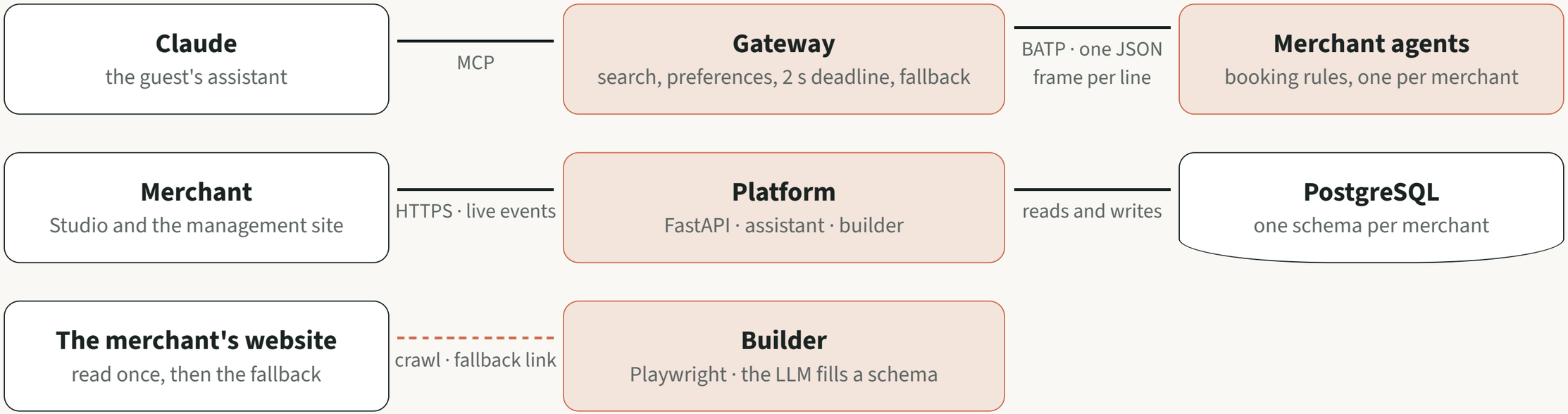
Pause the agent: guests get the booking page. Resume

bonus

A booking from the site's own form shows up live too

ARCHITECTURE

One connector for Claude, one agent per merchant



A booking reaches the merchant's screen in under 2 s: PostgreSQL's LISTEN / NOTIFY, then server-sent events.
Python 3.12 · MCP SDK v2 · FastAPI · PostgreSQL 16 · React · our own agent protocol, BATP, with MCP as Claude's bridge. Every choice has its ADR in `documentation/adr/`.

WHY IT HOLDS

Built so that a demo can't double-book

The guest says yes	Claude shows what it will book and waits for a yes. Every write carries an idempotency key, so a retry never books twice.	#14 · #109
One table, one time	A Postgres exclusion constraint on table and time range. Two bookings at once for the last table: exactly one wins.	ADR-0001
The LLM fills a form	It never writes code, SQL or tools. Its answer is validated, and every assumption is shown to the merchant.	ADR-0003
Never a dead end	A slow or paused agent, or a group too large: the guest gets the restaurant's own booking page.	§8.4
Only the owner	A secret link opens the management site. Only its hash is stored, and new builds need an invitation.	ADR-0018

TESTED FOR REAL

What runs on every pull request

1,500+

Python tests

On a real PostgreSQL through testcontainers. The booking rules, the constraint and the triggers are tested for real, not mocked.

2

Whole-stack runs

The demo in its order over HTTP, and a merchant's first evening in Chromium. Only the model and the crawl are scripted.

AA

Accessibility

WCAG 2.2 AA with axe-core and the keyboard, on every page and at both widths.

24

Screenshots

Every page and Studio state, compared pixel by pixel in a fixed browser image.

WHAT'S NEXT

From one city of restaurants to any merchant

Product

Ranking when many restaurants match: distance, availability and preferences. Proving that the person onboarding owns the website. Working with the booking tools merchants already use.

Reach

More verticals behind the same protocol: hairdressers, doctors, workshops. Claude finishing the website's own form when there is no agent. A business model: per booking, or a subscription.

QUESTIONS

Merci.

github.com/MaxEntz/BaselHack2026 · `documentation/ARCHITECTURE.md`